

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA1402
COMPILER DESIGN

COURSE NAME:

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS:4

Total Marks:50

Annexure A

DESIGN TASK

Code of Conduct:

I HARI CHARAN P(192511107)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

SIGNATURE OF THE STUDENT
COURSE FACULTY

PROGRAM 1:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
char input[50];
```

```
char stack[50];
```

```
int top = -1;
```

```
int ip = 0;
```

```
void push(char c) {
```

```
    stack[++top] = c;
```

```
}
```

```
char pop() {
```

```
    return stack[top--];
```

```
}
```

```
char peek() {
```

```
    return stack[top];
```

```
}
```

```
void printStack() {
```

```
    for (int i = 0; i <= top; i++)
```

```
        printf("%c", stack[i]);
```

```
}
```

```
int main() {
```

```
    printf("Grammar:\n");
```

```
printf("E->TA\n");
printf("A->+TA|e\n");
printf("T->FB\n");
printf("B->*FB|e\n");
printf("F->(E)|i\n\n");
```

```
printf("Enter input (use i for id): ");
scanf("%s", input);
```

```
strcat(input, "$");
```

```
push('$');
push('E');
```

```
printf("\nStack\t\tInput\t\tAction\n");
```

```
while (top != -1) {
```

```
    printStack();
    printf("\t\t%s\t\t", input + ip);
```

```
    char x = peek();
    char a = input[ip];
```

```
    if (x == a && x == '$') {
        printf("Accepted\n");
        break;
    }
```

```
else if (x == 'i' || x == '+' || x == '*' || x == '(' || x == ')' || x == '$') {
```

```
    if (x == a) {
```

```
        pop();
```

```
        ip++;
```

```
        printf("Match %c\n", a);
```

```
    } else {
```

```
        printf("Error\n");
```

```
        break;
```

```
    }
```

```
}
```

```
else if (x == 'E') {
```

```
    if (a == 'i' || a == '(') {
```

```
        pop();
```

```
        push('A');
```

```
        push('T');
```

```
        printf("E->TA\n");
```

```
    } else {
```

```
        printf("Error\n");
```

```
        break;
```

```
    }
```

```
}
```

```
else if (x == 'A') {
```

```
    if (a == '+') {
```

```
        pop();
```

```
        push('A');
```

```
        push('T');
```

```
        push('+');
        printf("A->+TA\n");
    } else if (a == ')' || a == '$') {
        pop();
        printf("A->e\n");
    } else {
        printf("Error\n");
        break;
    }
}
```

```
else if (x == 'T') {
    if (a == 'i' || a == '(') {
        pop();
        push('B');
        push('F');
        printf("T->FB\n");
    } else {
        printf("Error\n");
        break;
    }
}
```

```
else if (x == 'B') {
    if (a == '*') {
        pop();
        push('B');
        push('F');
        push('*');
    }
}
```

```

        printf("B->*FB\n");
    } else if (a == '+' || a == ')' || a == '$') {
        pop();
        printf("B->e\n");
    } else {
        printf("Error\n");
        break;
    }
}

else if (x == 'F') {
    if (a == 'i') {
        pop();
        push('i');
        printf("F->i\n");
    } else if (a == '(') {
        pop();
        push(')');
        push('E');
        push('(');
        printf("F->(E)\n");
    } else {
        printf("Error\n");
        break;
    }
}

}

return 0;

```

}

Output:

```
Enter input (use i for id): i+i*i

Stack      Input      Action
$E          i+i*i$     E->TA
$AT          i+i*i$     T->FB
$ABF          i+i*i$     F->i
$ABi          i+i*i$     Match i
$AB          +i*i$      B->e
$A           +i*i$      A->+TA
$AT+          +i*i$      Match +
$AT           i*i$      T->FB
$ABF          i*i$      F->i
$ABi          i*i$      Match i
$AB           *i$      B->*FB
$ABF*         *i$      Match *
$ABF          i$       F->i
$ABi          i$       Match i
$AB           $        B->e
$A            $        A->e
$             $        Accepted
```

PROGRAM 2:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
char input[50], stack[50];
```

```
int top = -1, i = 0;
```

```
void printState(char action[]) {
```

```
    printf("%-15s %-15s %s\n", stack, input + i, action);
```

```
}
```

```
int main() {
```

```
    printf("Grammar:\n");
```

```
    printf("E -> E+E\n");
```

```
    printf("E -> E*E\n");
```

```
    printf("E -> i\n");
```

```

printf("Enter input (use i for id): ");
scanf("%s", input);

strcat(input, "$");

printf("\n%-15s %-15s %s\n", "Stack", "Input", "Action");

while (1) {

    // Shift
    stack[++top] = input[i++];
    stack[top + 1] = '\0';
    printState("Shift");

    // Reduce i -> E
    if (top >= 0 && stack[top] == 'i') {
        stack[top] = 'E';
        printState("Reduce E->i");
    }

    // Reduce E*E -> E (Higher precedence)
    while (top >= 2 &&
           stack[top] == 'E' &&
           stack[top - 1] == '*' &&
           stack[top - 2] == 'E') {

        top -= 2;
        stack[top] = 'E';
        stack[top + 1] = '\0';
    }
}

```

```

        printState("Reduce E->E*E");
    }

// Reduce E+E -> E
while (top >= 2 &&
        stack[top] == 'E' &&
        stack[top - 1] == '+' &&
        stack[top - 2] == 'E') {

    // Delay reduction if next input is '*'
    if (input[i] == '*')
        break;

    top -= 2;
    stack[top] = 'E';
    stack[top + 1] = '\0';
    printState("Reduce E->E+E");
}

// Accept
if (strcmp(stack, "E") == 0 && input[i] == '$') {
    printState("Accept");
    printf("\nString Accepted\n");
    break;
}

if (input[i] == '$' && strcmp(stack, "E") != 0) {
    while (top >= 2 &&
            stack[top] == 'E' &&

```

```

        stack[top - 1] == '+' &&
        stack[top - 2] == 'E') {

    top -= 2;
    stack[top] = 'E';
    stack[top + 1] = '\0';
    printState("Reduce E->E+E");
}

if (strcmp(stack, "E") == 0) {
    printState("Accept");
    printf("\nString Accepted\n");
} else {
    printf("\nString Rejected\n");
}
break;
}
}

return 0;
}

```

OUTPUT:

Enter input (use i for id): i+i*i

Stack	Input	Action
i	+i*i\$	Shift
E	+i*i\$	Reduce E->i
E+	i*i\$	Shift
E+i	*i\$	Shift
E+E	*i\$	Reduce E->i
E+E*	i\$	Shift
E+E*i	\$	Shift
E+E*i	\$	Reduce E->i
E+E	\$	Reduce E->E*i
E	\$	Reduce E->E+E
E	\$	Accept

String Accepted

PROGRAM 3:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
char input[100];
```

```
char stack[100];
```

```
int top = -1, ip = 0;
```

```
void push(char x)
```

```
{
```

```
    stack[++top] = x;
```

```
}
```

```
void pop()
```

```
{
```

```
    top--;
```

```
}
```

```
void displayStack()
```

```
{
```

```
    for (int i = 0; i <= top; i++)
```

```

        printf("%c", stack[i]);
    }

int main()
{
    printf("Grammar after removing Left Recursion:\n");
    printf("S -> TA\n");
    printf("A -> +TA | e\n");
    printf("T -> FB\n");
    printf("B -> *FB | e\n");
    printf("F -> (S) | i\n\n");

    printf("Enter input (use i for id): ");
    scanf("%s", input);

    strcat(input, "$");

    push('$');
    push('S');

    printf("\n%-15s %-15s %s\n", "Stack", "Input", "Action");

    while (top >= 0)
    {
        displayStack();
        printf("\t\t%s\t\t", input + ip);

        char X = stack[top];
        char a = input[ip];
    }
}

```

```

// Accept
if (X == '$' && a == '$')
{
    printf("Accepted\n");
    printf("\nString Accepted\n");
    break;
}

// Match terminals
if (X == 'i' || X == '+' || X == '*' || X == '(' || X == ')' || X == '$')
{
    if (X == a)
    {
        pop();
        ip++;
        printf("Match %c\n", a);
    }
    else
    {
        printf("Error\n");
        break;
    }
}

// S -> TA
else if (X == 'S')
{
    if (a == 'i' || a == '(')

```

```
{
    pop();
    push('A');
    push('T');
    printf("S->TA\n");
}
else
{
    printf("Error\n");
    break;
}
}
```

```
// A -> +TA | e
```

```
else if (X == 'A')
```

```
{
    if (a == '+')
    {
        pop();
        push('A');
        push('T');
        push('+');
        printf("A->+TA\n");
    }
    else if (a == ')' || a == '$')
    {
        pop();
        printf("A->e\n");
    }
}
```

```
    else
    {
        printf("Error\n");
        break;
    }
}
```

```
// T -> FB
```

```
else if (X == 'T')
{
    if (a == 'i' || a == '(')
    {
        pop();
        push('B');
        push('F');
        printf("T->FB\n");
    }
    else
    {
        printf("Error\n");
        break;
    }
}
```

```
// B -> *FB | e
```

```
else if (X == 'B')
{
    if (a == '*')
    {
```

```
        pop();
        push('B');
        push('F');
        push('*');
        printf("B->*FB\n");
    }
    else if (a == '+' || a == ')' || a == '$')
    {
        pop();
        printf("B->e\n");
    }
    else
    {
        printf("Error\n");
        break;
    }
}
```

```
// F -> i | (S)
else if (X == 'F')
{
    if (a == 'i')
    {
        pop();
        push('i');
        printf("F->i\n");
    }
    else if (a == '(')
    {
```

```
        pop();
        push(')');
        push('S');
        push('(');
        printf("F->(S)\n");
    }
    else
    {
        printf("Error\n");
        break;
    }
}

return 0;
}
```

OUTPUT:

Enter input (use i for id): i+i*i

Stack	Input	Action
\$S	i+i*i\$	S → TA
\$AT	i+i*i\$	T → FB
\$ABF	i+i*i\$	F → i
\$ABi	i+i*i\$	Match i
\$AB	+i*i\$	B → e
\$A	+i*i\$	A → +TA
\$AT+	+i*i\$	Match +
\$AT	i*i\$	T → FB
\$ABF	i*i\$	F → i
\$ABi	i*i\$	Match i
\$AB	*i\$	B → *FB
\$ABF*	*i\$	Match *
\$ABF	i\$	F → i
\$ABi	i\$	Match i
\$AB	\$	B → e
\$A	\$	A → e
\$	\$	Accepted

String Accepted

PROGRAM 4:

★ DESIGNING AN UNAMBIGUOUS GRAMMAR WITH CUSTOM OPERATORS (Q11)

TASK: Create CFG for %, ^, +, -, with User-Defined Precedence/Associativity & Construct Parser

